

Technische Dokumentation – InnoShop

Version: 1.0
Datum: 19.08.2025
Autor(en): Ashkan Mashayekh
Zweck: InnoShop ist ein webbasiertes E-Commerce-System zur Verwaltung von Produkten, Bestellungen und Benutzern. Es erlaubt Kunden, Produkte zu kaufen, Wunschlisten zu erstellen und Bestellungen aufzugeben, während Administratoren den Shop verwalten können.

1. Systemvoraussetzungen

Komponente	Technologie
Java-Version	Java 17
Build-Tool	Maven
UI-Framework	Vaadin 24.7.6
Backend	JPA (Hibernate)
Datenbank	PostgreSQL
Server	Jetty (lokal), WAR-Deploy
Sonstiges	Git

2. Architektur & Projektstruktur

2.1 Architekturübersicht

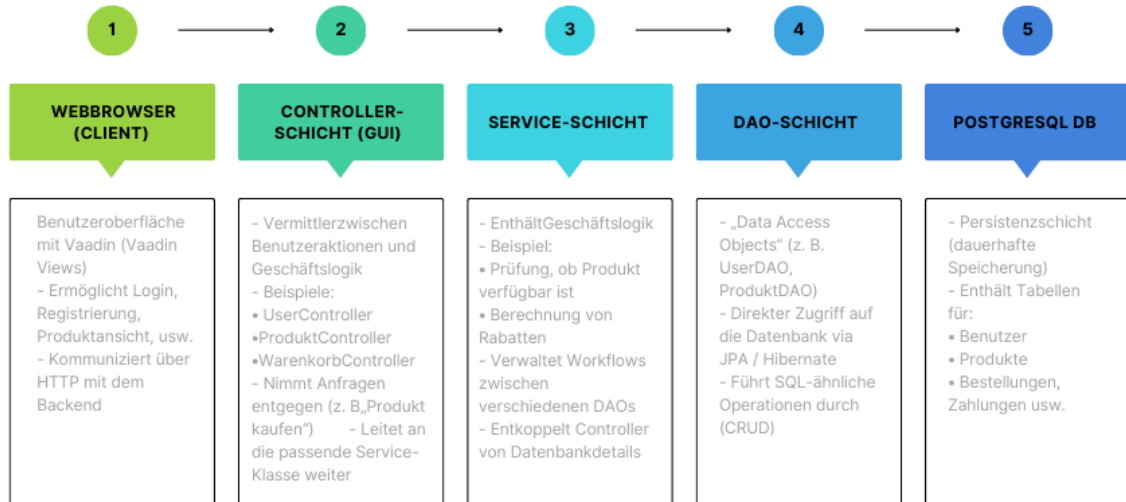
InnoShop verwendet eine klassische MVC-Architektur mit klarer Trennung von:

- Frontend: Vaadin-WebUI (Client-seitig)
- Backend: Java, JPA/Hibernate (Datenzugriff & Geschäftslogik)
- Persistenz: PostgreSQL
- Deployment: als WAR auf Jetty oder Tomcat

Architekturprinzipien:

- Trennung von Darstellung, Logik und Datenhaltung
- Modularität und Wartbarkeit

Architekturdiagramm



2.2 Projektstruktur (Treemodell)

InnoShop/

```

├── pom.xml      ← Maven-Projektdatei
├── README.md    ← Projektdokumentation
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── org/commercetron/
│   │   │   │   ├── beans/      ← Entity-Klassen (Admin, User, Produkt usw.)
│   │   │   │   ├── cli/        ← Konsolenmenüs (z. B. Login, Registrierung)
│   │   │   │   ├── controller/ ← Controller für Geschäftslogik
│   │   │   │   ├── dao/        ← Datenbankzugriff (DAO-Klassen)
│   │   │   │   └── gui/        ← UI-Komponenten mit Vaadin

```

```
| | └─ interface/ ← Interfaces für DAO/Controller
| | └─ utils/ ← Hilfsklassen und Main
| └─ resources/
| | └─ META-INF/
| | └─ persistence.xml ← JPA-Konfiguration
| └─ webapp/
| └─ WEN-INF/
| └─ web.xml ← Servlet-Konfiguration
```

3. Installation & Konfiguration

Voraussetzungen:

- Java 17
- Maven installiert
- PostgreSQL-Datenbank (z. B. lokal oder in Docker)

Schritte:

```
git clone https://github.com/dein-repo/innoshop.git
```

```
cd innoshop
```

```
mvn jetty:run
```

Konfiguration:

In src/main/resources/application.properties:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/innoshop
```

```
spring.datasource.username=postgres
```

```
spring.datasource.password=deinpasswort
```

4. Ablaufdiagramm: Bestellprozess

🔴 Hinweis: Zahlungs- und Versandprozesse sind aktuell simuliert, aber im Code strukturell vorbereitet (für echte API-Anbindungen).

5. GUI-Design

Ansichten:

- Login / Registrierung
- Produktkatalog (mit Filter/Suche)
- Warenkorb & Wunschliste

- Checkout-Prozess
- Admin-Panel (Produkt- & Benutzerverwaltung)

Designprinzipien:

- Intuitives Layout (Vaadin-Layout-Komponenten)
- Responsives Design (automatisch durch Vaadin)
- Formularvalidierung (mittels Bean Validation / JSR-380)

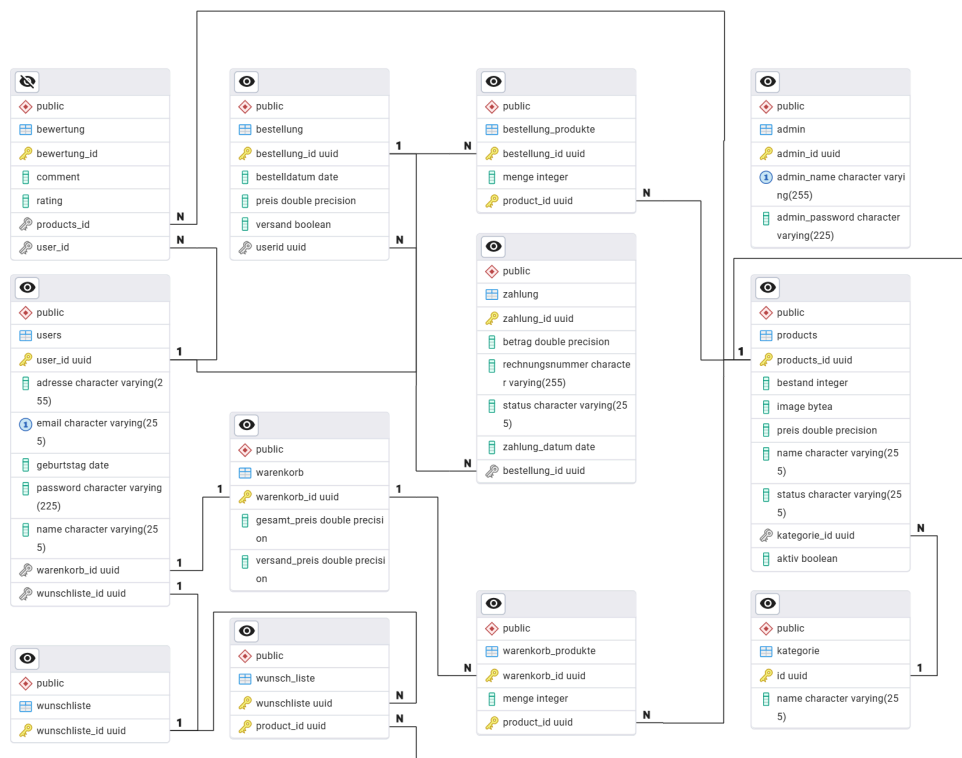
6. Datenmodell (vereinfacht)

Entitäten:

- Benutzer (id, name, email, passwort, rolle)
- Produkt (id, name, preis, kategorie)
- Bestellung (id, benutzer, produkte, status)
- Zahlung (id, betrag, status)
- Warenkorb (id, benutzer_id)
- Wunschliste (id, benutzer_id)

Beziehungen:

- Benutzer 1 ↔ n Bestellungen
- Benutzer 1 ↔ 1 Warenkorb
- Benutzer 1 ↔ 1 Wunschliste
- Warenkorb n ↔ m Produkte
- Wunschliste n ↔ m Produkte
- Bestellung n ↔ m Produkte
- Bestellung 1 ↔ n Zahlung



7. Funktionale Anforderungen (F1–F12)

ID	Beschreibung
F1	Nutzerkonto erstellen
F2	Login / Logout
F3	Produktkatalog anzeigen
F4	Produkte filtern/suchen
F5	Produkte in Warenkorb legen
F6	Wunschliste verwalten
F7	Bestellung aufgeben
F8	Zahlung abwickeln (simuliert)
F9	Versandstatus verfolgen (simuliert)
F10	Bewertungen schreiben/lesen
F11	Bestellhistorie anzeigen
F12	Admin-Login & Verwaltung

8. Nicht-funktionale Anforderungen

- **Performance:** Antwortzeiten < 2s bei bis zu 100 gleichzeitigen Benutzern
- **Sicherheit:**
 - Passwort-Hashing (z. B. SHA-256 oder BCrypt)
 - Rollenbasierte Zugriffskontrolle (Admin/Nutzer)
- **Skalierbarkeit:** Modularer Aufbau zur Erweiterung (z. B. externe APIs)
- **Portabilität:** Containerisierung via Docker möglich

9. Schnittstellen & Integration

- **JPA / Hibernate:** für ORM-Datenzugriff
- **REST-API (geplant):** für externe Services (Zahlung/Versand)
- **Mögliche APIs zur Anbindung:**
 - Stripe / PayPal für Zahlungsabwicklung
 - DHL API für Sendungsverfolgung

10. Tests & Qualitätssicherung

Testart	Beschreibung
Unit-Tests	Service-Methoden, Validierungen (JUnit 5)
Integrationstests	DB-Zugriffe via Testcontainers
UI-Tests	Vaadin TestBench / Selenium (optional)
CI/CD	GitHub Actions für Build & Test

Beispieltests:

- Benutzerregistrierung mit gültigen/ungültigen Eingaben
- Checkout mit leerem Warenkorb
- Admin kann Produkte bearbeiten

11. Deployment

Umgebung	Konfiguration
Lokal	mvn jetty:run (Port 9090)
Server	WAR-Deployment auf Tomcat oder Jetty
DB	PostgreSQL (lokal oder Container-basiert)

12. Roadmap & Erweiterungspotenziale

-  Integration echter Zahlungsanbieter (z. B. Stripe, PayPal)
-  Anbindung von Versanddienstleistern (z. B. DHL, UPS)
-  SOON Mobile App (Umsetzung mit Flutter oder React Native)
-  SOON Empfehlungssystem (z. B. Collaborative Filtering zur Produktempfehlung)
-  SOON REST-API für die Integration von Drittsystemen

Hinweis:

Derzeit sind Zahlungs- und Versandprozesse noch simuliert. Im Quellcode wurde jedoch bereits eine modulare Struktur geschaffen, sodass eine spätere Anbindung echter externer APIs ohne tiefgreifende Änderungen möglich ist.

13. Versionsverlauf

Version	Datum	Änderungen
1.0	19.08.2025	Erste vollständige Version

✓ Fazit

InnoShop bietet ein solides Grundgerüst für ein modernes Online-Shopsystem. Die klare Architektur, die Nutzung bewährter Technologien (Vaadin, JPA, Maven, PostgreSQL) sowie die Erweiterbarkeit machen es zu einer guten Basis für zukünftige Entwicklungen.